

1. Arquitetura de software é o esqueleto do sistema. Ela define como os diferentes componentes do sistema se conectam e interagem. É importante porque garante que o sistema seja bem organizado, fácil de manter e pronto para crescer, além de ajudar a evitar problemas futuros, como dificuldades de atualização ou falhas de desempenho.

2. Sim, concordo com a frase. Requisitos não funcionais, como desempenho, segurança e escalabilidade, influenciam diretamente as decisões de design. Por exemplo, se o sistema precisa ser rápido, o desenho precisa prever soluções que garantam essa agilidade. Portanto, o design do sistema deve atender a esses requisitos.

3. Ao criar uma arquitetura de software, você deve considerar:

- **Desempenho:** O sistema precisa ser rápido?
- **Segurança:** Como os dados serão protegidos?
- **Escalabilidade:** O sistema vai crescer sem perder qualidade?
- **Manutenção:** Será fácil de atualizar e corrigir? Esses pontos garantem que o sistema funcione bem agora e no futuro.

4. Design patterns são soluções comuns para problemas frequentes no desenvolvimento. Eles ajudam a resolver desafios de design de forma eficiente, já que são soluções testadas e aprovadas. Usar patterns facilita o desenvolvimento e torna o código mais claro e fácil de manter.

5.

- **Coesão:** Uma parte do sistema faz uma única coisa bem. Quanto mais focado for um módulo, melhor.
- **Acoplamento:** Refere-se a quanto as partes do sistema dependem umas das outras. O ideal é que os módulos dependam o mínimo possível entre si.

Esses conceitos são essenciais para uma boa arquitetura, pois garantem que o sistema seja fácil de modificar sem danificar nada.

6. Para um sistema de e-commerce, a divisão lógica em pacotes pode ser assim:

- **Interface de Usuário (UI):** Páginas e formulários.
- **Serviços de Negócio:** Regras e processos de compras.
- **Banco de Dados:** Armazenamento das informações de clientes e produtos.
- **Segurança:** Autenticação e autorização dos usuários.

Essa divisão organiza o código e facilita a manutenção.

**7.** O design de software afeta diretamente a experiência do usuário (UX). Um sistema bem projetado melhora a usabilidade, tornando-o mais intuitivo e rápido. Se o design não for bem feito, o usuário pode ter dificuldades e frustrações ao usar o sistema.

**8.** A UML (Unified Modeling Language) é usada para visualizar e planejar como o software será construído. Exemplos de diagramas:

- **Diagrama de Classes:** Mostra como as classes do sistema se relacionam.
- **Diagrama de Casos de Uso:** Define as funcionalidades que o sistema oferece aos usuários.

Esses diagramas ajudam a organizar e comunicar as ideias de design.

**9.** Ao integrar sistemas antigos (legados) com um novo software, você precisa garantir que eles possam trocar dados e trabalhar juntos. Isso pode ser feito criando APIs, adaptadores para traduzir dados ou migrando partes do sistema antigo aos poucos, mantendo tudo funcionando.

**10.** A escolha das tecnologias afeta diretamente o design. Por exemplo, usar uma arquitetura de micros serviços muda a forma de organizar os componentes do sistema. Escolhas ruins podem comprometer o desempenho e a capacidade de expansão do software, então a tecnologia precisa estar alinhada com as necessidades do sistema.